141. Linked List Cycle Easy

Acceptance: 53.7% | LeetCode

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter**.

Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:

```
**Input:** head = [3,2,0,-4], pos = 1
**Output:** true
**Explanation:** There is a cycle in the linked list,
where the tail connects to the 1st node (0-indexed).
```

Example 2:

```
**Input:** head = [1,2], pos = 0
**Output:** true
**Explanation:** There is a cycle in the linked list,
where the tail connects to the 0th node.
```

Example 3:

```
**Input:** head = [1], pos = -1
**Output:** false
**Explanation:** There is no cycle in the linked list.
```

Constraints:

- The number of the nodes in the list is in the range `[0, 104]`.
- `-105 <= Node.val <= 105`
- `pos` is `-1` or a **valid index** in the linked-list.

Follow up: Can you solve it using `O(1)` (i.e. constant) memory?

Claude Opus Solution

Linked List Cycle

1. Intuition

The key insight is **Floyd's Cycle Detection Algorithm** (also called the "Tortoise and Hare" algorithm).

Imagine two runners on a circular track moving at different speeds. The faster runner will **eventually lap** the slower one — they'll meet again. The same logic applies here:

- Use **two pointers** moving at different speeds: `slow` moves 1 step, `fast` moves 2 steps.
- **If there's no cycle**, the `fast` pointer will reach the end (`None`) and we stop.
- **If there's a cycle**, the `fast` pointer will keep looping and eventually "catch up" to `slow` from behind — they will meet at the same node.

Why must they meet? Once both pointers are inside the cycle, the gap between them decreases by exactly 1 each step (fast gains 2, slow gains 1 → net gain of 1). Since the gap shrinks by 1 every iteration in a bounded loop, it must eventually become 0 (they meet).

---

2. Approach (Pseudocode)

```
function hasCycle(head):
    slow = head
    fast = head

    # Loop while fast can take two steps (fast and fast.next must exist)
    while fast is not None AND fast.next is not None:
        slow = slow.next          # move slow 1 step
        fast = fast.next.next     # move fast 2 steps

        if slow == fast:          # pointers met → cycle exists
            return True

    # fast reached the end → no cycle
    return False
```

---

3. Python Solution

```python
from typing import Optional


class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Initialize both pointers at the head.
        # 'slow' advances one node per iteration (tortoise).
        # 'fast' advances two nodes per iteration (hare).
        slow = fast = head

        # Continue while the fast pointer can safely take 2 steps.
        # - 'fast' must exist to take its first step.
        # - 'fast.next' must exist to take its second step.
        # If either is None, we've hit the end → no cycle.
        while fast and fast.next:
            slow = slow.next          # tortoise moves 1 step
            fast = fast.next.next     # hare moves 2 steps

            # If the two pointers reference the SAME node (by identity),
            # the hare has lapped the tortoise → a cycle exists.
            if slow is fast:
                return True

        # Fast pointer fell off the end of the list → no cycle.
        return False
```

> **Note:** I use `is` instead of `==` because we're comparing node *identity* (same object in memory), not value. The original reference uses `==`, which works for node objects too since `ListNode` doesn't override `__eq__`, but `is` is more explicit and correct here.

---

4. Complexity Analysis

Time Complexity: O(n) where `n` is the number of nodes. - **No cycle:** `fast` reaches the end in `n/2` steps → O(n). - **With cycle:** The slow pointer travels at most `n` steps before the pointers meet. Once both are in the cycle (of length `L`), the gap closes by 1 each step, so they meet within at most `L` additional iterations. Total work is still bounded by O(n).

Space Complexity: O(1) - We only use two pointer variables regardless of input size. This satisfies the follow-up's constant-memory requirement.

---

5. Alternative Approaches

A) Hash Set (visited nodes) — O(n) time, O(n) space

Store each visited node in a set. If we encounter a node already in the set, there's a cycle.

```python
def hasCycle(self, head: Optional[ListNode]) -> bool:
    seen = set()
    while head:
        if head in seen:    # revisiting a node → cycle
            return True
        seen.add(head)
        head = head.next
    return False
```

Trade-off: Simpler to reason about, but uses O(n) extra memory — **fails the follow-up constraint**.

B) Node Mutation (destructive) — O(n) time, O(1) space

Mark each visited node by changing its value (or `next`) to a sentinel. Detect a cycle if you reach a marked node. **Not recommended** since it destroys the input list and may break value constraints.

Pattern Summary

This problem is the classic introduction to the **Fast & Slow Pointer** pattern. The same technique powers related problems: - **LC 142** – Linked List Cycle II (find the *start* of the cycle). - **LC 876** – Middle of the Linked List. - **LC 287** – Find the Duplicate Number (cycle in an array-as-linked-list).

When you see "detect a cycle" or "find the middle" in a linked structure with an O(1) space requirement, reach for two pointers moving at different speeds.

Codex Solution

1. Intuition

The key pattern is **Floyd's Cycle Detection Algorithm**, also called the **slow and fast pointer technique**.

We use two pointers:

- `slow` moves **1 step** at a time.
- `fast` moves **2 steps** at a time.

If the linked list has **no cycle**, the `fast` pointer will eventually reach `None`.

If the linked list **has a cycle**, the `fast` pointer will eventually "lap" the `slow` pointer inside the cycle, meaning both pointers will point to the same node.

This is similar to two runners on a circular track: the faster runner will eventually catch up to the slower runner.

2. Approach

Pseudocode

```
function hasCycle(head):
    slow = head
    fast = head

    while fast is not null and fast.next is not null:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return true

    return false
```

Explanation

1. Start both `slow` and `fast` at the head.
2. Move `slow` by one node.
3. Move `fast` by two nodes.
4. If they ever meet, there is a cycle.
5. If `fast` reaches the end of the list, there is no cycle.

3. Python Solution

```python
from typing import Optional


# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        """
```

```
        Detect whether a linked list has a cycle using Floyd's Cycle Detection.

        slow moves one step at a time.
        fast moves two steps at a time.

        If there is a cycle, fast will eventually meet slow.
        If there is no cycle, fast will eventually reach None.
        """

        # Initialize both pointers at the head of the linked list
        slow = head
        fast = head

        # Continue while fast can still move two steps
        while fast is not None and fast.next is not None:
            # Move slow pointer one step
            slow = slow.next

            # Move fast pointer two steps
            fast = fast.next.next

            # If both pointers point to the same node,
            # then a cycle exists
            if slow is fast:
                return True

        # If fast reaches the end, there is no cycle
        return False
```

---

4. Complexity Analysis

Time Complexity: `O(n)`

Each pointer visits at most `O(n)` nodes.

- If there is no cycle, the `fast` pointer reaches the end of the list.
- If there is a cycle, the `fast` pointer catches the `slow` pointer after a finite number of steps.

Therefore, the time complexity is:

```
O(n)
```

where `n` is the number of nodes in the linked list.

Space Complexity: `O(1)`

We only use two pointers:

```
slow
fast
```

No extra data structures are used.

Therefore, the space complexity is:

```
O(1)
```

This satisfies the follow-up requirement.

---

5. Alternative Approaches

1. Hash Set Approach

We can store every visited node in a set.

If we encounter a node that is already in the set, then there is a cycle.

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        visited = set()

        current = head
```

```
        while current is not None:
            if current in visited:
                return True

            visited.add(current)
            current = current.next

        return False
```

Complexity

- Time Complexity: `O(n)`
- Space Complexity: `O(n)`

This approach is simpler to understand but does not satisfy the `O(1)` memory follow-up.

---

2. Modifying Nodes Approach

Another possible approach is to mark nodes as visited by modifying them.

However, this is usually not recommended because:

- It changes the input linked list.
- It may not be allowed in interviews or online judges.
- It can cause side effects if the list is used later.

Key Insight

Use **two pointers moving at different speeds**.

If there is a cycle, they must eventually meet.
If there is no cycle, the faster pointer reaches the end.